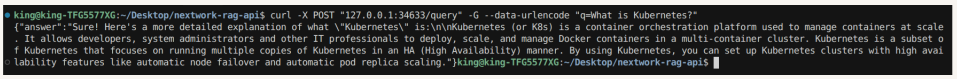
Deploy a RAG API to Kubernetes

Ahmed Tetteh



Ahmed Tetteh
NextWork Student

Introducing Today's Project!

In this project, I will deploy my containerized RAG API to Kubernetes. I'm doing this project to learn how a kubernetes tool such as Minikube is used in local development of containerized applications. Kubernetes will help me manage my containerized RAG API by restarting the container if it crashes, network the containers and keep my application running during updates.

Key services and concepts

Tools I used were AI model (Tinyllama), Kubernetes, Minikube, kubectl, Docker, FastAPI, RAG (Retrieval-Augmented Generation)Key concepts I learnt include Kubernetes self-healing, Deployments, using Minikube to provision a kubernetes cluster.

Challenges and wins

This project took me approximately 4 hours. The most challenging part was troubleshooting the Internal error when my API couldn't reach Ollama on my host machine. It was most rewarding to the API generate AI responses even from within a cluster.

Ahmed Tetteh
NextWork Student

Why I did this project

I did this project because I wanted to learn about kubernetes deployments using tools like Minikube and Docker. One thing I'll apply from this is how to deploy and access applications from within a kubernetes cluster.

Ahmed Tetteh
NextWork Student

Setting Up My Docker Image

In this step, I'm setting up my docker image containing everything I need for my RAG API to work smoothly. I need a Docker image because Kubernetes will use that image to create, run and manage containers.

```
king@king-TFG5577XG:~$ docker images
REPOSITORY          TAG       IMAGE ID       CREATED         SIZE
ahmed3015/rag-api   latest    aee8ce62541f   36 hours ago    1.36GB
```

What the Docker image contains

I check I had a Docker image by running the command "docker images". This Docker image contains all the python packages, application code and files used by the RAG API.

Ahmed Tetteh
NextWork Student

Installing Kubernetes Tools

In this step, I'm installing both Minikube and Kubectl. I need these tools because Minikube is the tool that will allow me to run a local kubernetes cluster on my computer, and Kubectl is the command line too for communicating with my Kubernetes cluster. Every command with the kubectl tool first goes through the api-server, and then the other components in the kubernetes environment run those commands.

```
king@king-TFG5577XG:~$ minikube version
minikube version: v1.37.0
commit: 65318f4cfff9c12cc87ec9eb8f4cdd57b25047f3
king@king-TFG5577XG:~$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

Verifying the tools are installed

I installed Minikube using brew package manager. I also installed kubectl using the brew package manager by running the command- "brew install kubectl".

I could tell both installations were successful by running the commands "minikube version" and "kubectl version --client"

Ahmed Tetteh
NextWork Student

Starting My Kubernetes Cluster

In this step, I'm starting my single node kubernestes cluster using Minikube. A Kubernetes cluster is a set of machines (nodes) that work together to manage containerized applications.

```
king@king-TFG5577XG:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

king@king-TFG5577XG:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   22m   v1.34.0
```

Loading the Docker image into Minikube

I started the cluster by running "minikube start" and saw Minikube downloaded all the components of kubernetes needed to run on my single node cluster. Then I ran "docker pull rag-api" to load my image locally. kubectl get nodes showed the status as of my nodes in the cluster, but since Minikube only creates a single node ccluster to run everything, only one node showed -named minikube

Ahmed Tetteh
NextWork Student

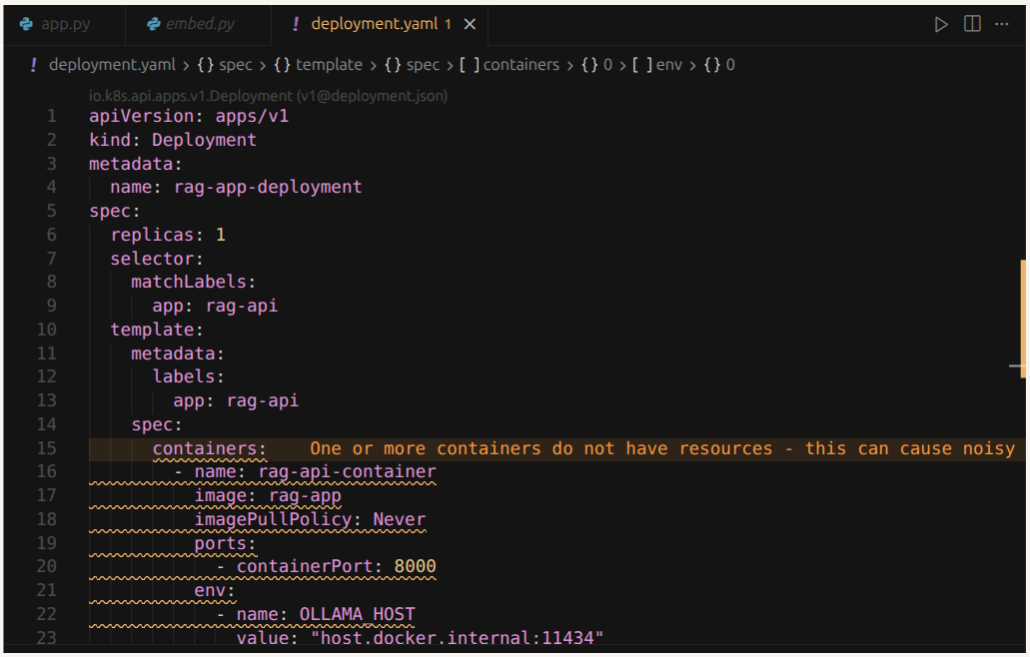
Why load image into Minikube

I needed to load my docker image into Minikube's docker storage because Minikube creates an isolated enviroment on my computer, hence, all processes, files, and Docker images are not automatically shared between them. Without this step, Kubernetes would would not be able to load my RAG API docker image and create containers from it. The eval minikube docker-env command helps by setting my environment variables to redirect the my docker CLI to use Minikube's docker daemon instead of my host machine's docker daemon.

Ahmed Tetteh
NextWork Student

Deploying to Kubernetes

In this step, I'm deploying my RAG PI via Kubernetes deployment. I need a Deployment because it will serve as the blueprint that tells kubernetes how I want to run my app.



How the Deployment keeps my app running

The deployment.yaml file tells Kubernetes how many containers of my application I want running at all times, which pods on the node to manage, and the image I want to use for my container. The key parts are apiVersion, kind, metadata and spec. The image field specifies the image I want to use to run my container, which in my case, is the rag-api docker image. The replicas field shows my desired number of pods I want running at all times.

```
king@king-TFG5577XG:~/Desktop/nextwork-rag-api$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
rag-app-deployment-86b57f8f77-q5mr2   1/1     Running   0          54s
```

What did you observe when checking your pods?

I ran kubectl get pods and saw the number of containers running in my pod, and how long my pod has been active. The pod had the name in the format of "Deployment name + ReplicaSet ID + unique ID" , and status. "RUNNING" which means the pod is active and running with the image specified. The READY column showed "1/1" which indicates 1 container ready out of 1 total in this Pod.

Ahmed Tetteh
NextWork Student

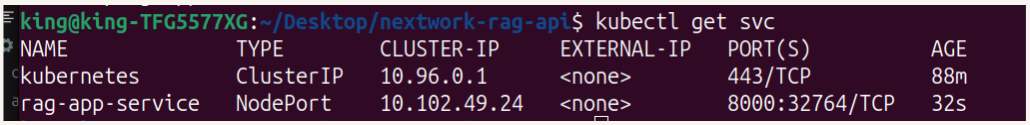
Creating a Service

In this step, I'm creating a Service to provide networking to my application. I need a Service because it will be provide a stable endpoint to reach my pods, load balancing and automatically route traffic to the desired pod. If a Service didn't exist our connection to pods would always be breaking because pods are ephemeral, so, are easily restarted with a new IP address.

What does the service.yaml file do?

The service.yaml file tells Kubernetes where to listen for traffic coming into the cluster, and which pods to route the traffic to. The selector finds Pods by matching the label of the pods specified in the service.yaml file. The port configuration allows port 8000 on the Service to connect to port 8000 on the Pod. NodePort enables access from outside by creating a port on the node to route traffic between them.

Ahmed Tetteh
NextWork Student



What kubectl commands did you run to create the service?

I applied my Service file by running "kubectl apply -f service.yaml". I then verified that the Service was created by running the command "kubectl get services". This showed me the port kubernetes has assigned for my Nodeport and other details.

Ahmed Tetteh
NextWork Student

Accessing My API Through Kubernetes

In this step, I'm testing access to my pods from the outside, that is, either via browser or the terminal.

How I accessed my API

I tested my API by running "curl -X POST "127.0.0.1:34633/query" -G --data-urlencode "q=What is Kubernetes?""The response showed an AI-generated response explaining what is Kubernetes. This confirms that my RAG APi still works fine even when containerized and running in a kubernetes cluster. The main difference between Docker and Kubernetes deployment is Docker runs apps in a container, while Kubernetes runs apps in a cluster, abstracting the container management away from us. It handles the container management on our behalf, making sure things are running smoothly.

Request flow through Kubernetes

Ahmed Tetteh
NextWork Student

Testing Self-Healing

In this project extension, I'm demonstrating container recovery using kubernetes's self healing. Self-healing is important because previously, when docker containers crash, we would need to manually start it back up again, but with self-healing, Kubernetes can actually start the container automatically when it crashes by matching the current state to the desired state. This can all be done without manual intervention.

```
king@king-TFG5577XG:~/Desktop/nextwork-rag-api$ kubectl get pods --watch
NAME                                 READY   STATUS    RESTARTS   AGE
rag-app-deployment-86b57f8f77-h9rxq   1/1     Running   0          52m
rag-app-deployment-86b57f8f77-h9rxq   1/1     Terminating   0          59m
rag-app-deployment-86b57f8f77-h9rxq   1/1     Terminating   0          59m
rag-app-deployment-86b57f8f77-2xmlr   0/1     Pending       0          0s
rag-app-deployment-86b57f8f77-2xmlr   0/1     Pending       0          0s
rag-app-deployment-86b57f8f77-2xmlr   0/1     ContainerCreating   0          0s
rag-app-deployment-86b57f8f77-2xmlr   1/1     Running             0          1s
rag-app-deployment-86b57f8f77-h9rxq   0/1     Completed           0          59m
rag-app-deployment-86b57f8f77-h9rxq   0/1     Completed           0          59m
rag-app-deployment-86b57f8f77-h9rxq   0/1     Completed           0          59m
```

What did you observe when you deleted the pod?

When I deleted the pod, I saw the status change from... Terminating to ContainerCreating to Running.

A new pod was created because Kubernetes detected that change in in pod state, and corrected this matching the desired state against the actual or current state. This is what what we called the reconcilation loop.

How the Service routed traffic to the new pod

The Service automatically routed traffic to the new pod because the new pod was created with the same label as the previous pod "rag-api". The Service has the same pod label in its selector field to connect to pods and route traffic to them. Without Kubernetes this would have caused the entire RAG API to be unaccessible and stopped working.

Ahmed Tetteh
NextWork Student
nextwork.org

Self-healing is critical in production because it keeps things running smoothly when there are impactful events like hardware failure, memory leaks (i.e, when apps have memory leaks leading to them crashing) and bugs introduced during deployment or updates. Kubernetes is one of the main reasons why servers at big tech companies such as Netflix and Spoitfy are always running even when conainers crash. Kubernetes automatically spins up new pods and containers to keep things running as expected.

The place to learn & showcase your skills

Check out nextwork.org for more projects